



Pashov Audit Group

Regnum Aurum Security Review

February 7th 2026 - February 16th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Regnum Aurum	4
5. Executive Summary	4
6. Findings	5
Low findings	6
[L-01] <code>claimProtocolFees</code> can be bricked if <code>pendingProtocolFee > buffer</code>	6
[L-02] <code>DebtToken._update()</code> emits duplicate <code>Transfer</code> event	7
[L-03] <code>rescueToken</code> does not protect vault share or reserve asset	7
[L-04] Mismatch between NatSpec and implementation for <code>liquidateBorrower</code> pause state	8
[L-05] Missing event emission on setter function	8
[L-06] <code>claimProtocolFees</code> lacks access control	8



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/contracts** repository was done by Pashov Audit Group, during which **montecristo**, **KlosMitSoss**, **Blockace**, **farismaulana**, **0x37** engaged to review **Regnum Aurum**. A total of **6** issues were uncovered.

Protocol Summary

Project Name	Regnum Aurum
Protocol Type	RWA tokenization
Timeline	February 7th 2026 - February 16th 2026

Review commit hash:

- [802a9e1fb7e4f7d618c1b2d0656e862e999e56cc](#)
(RegnumAurumAcquisitionCorp/contracts)

Fixes review commit hash:

- [2c684bff7a4808be54e5c8139e149ed436f8039b](#)
(RegnumAurumAcquisitionCorp/contracts)

Scope

DebtToken.sol RToken.sol DEToken.sol ERC20AssetAdapter.sol
ERC721AssetAdapter.sol LendingPool.sol LendingPoolStorage.sol
LiquidationProxy.sol VaultProxy.sol LiquidationStrategyProxy.sol
LiquidationSwap.sol StabilityPool.sol StabilityPoolStorage.sol
ReserveLibrary.sol



6. Findings

Findings count

Severity	Amount
Low	6
Total findings	6

Summary of findings

ID	Title	Severity	Status
[L-01]	<code>claimProtocolFees</code> can be bricked if <code>pendingProtocolFee > buffer</code>	Low	Resolved
[L-02]	<code>DebtToken._update()</code> emits duplicate <code>Transfer</code> event	Low	Resolved
[L-03]	<code>rescueToken</code> does not protect vault share or reserve asset	Low	Resolved
[L-04]	Mismatch between NatSpec and implementation for <code>liquidateBorrower</code> pause state	Low	Resolved
[L-05]	Missing event emission on setter function	Low	Resolved
[L-06]	<code>claimProtocolFees</code> lacks access control	Low	Resolved



Low findings

[L-01] `claimProtocolFees` can be bricked if `pendingProtocolFee > buffer`

`LendingPool.claimProtocolFees` transfers pending fee amount from `RToken` to `LendingPool`.

```
27:     function claimProtocolFees(ReserveLibrary.ReserveData storage reserve, address
feeCollector) external returns (address tokenAddress, uint256 claimedAmount) {
...
40:@>     IRToken(reserveRTokenAddress).transferAsset(address(this), pendingAmount);
41:     bool approved = IERC20(reserveAssetAddress).approve(feeCollector, pendingAmount);
42:     if (!approved) revert ApprovalFailed();
43:     IBaseCollector(feeCollector).collectFee(reserveAssetAddress, address(this),
pendingAmount, keccak256("PROTOCOL_FEE"));
44: }
```

However, if current `reserve.totalLiquidity` is low, then `RToken` may not hold enough `crvusd` because `desiredBuffer < pendingProtocolFee`.

As a result, `FeeCollector.collectFee` may revert with `ERC20InsufficientBalance` error.

Recommendation

```
diff --git a/contracts/core/pools/LendingPool/LendingPool.sol b/contracts/core/pools/
LendingPool/LendingPool.sol
index e343789..6ed8809 100644

--- a/contracts/core/pools/LendingPool/LendingPool.sol
+++ b/contracts/core/pools/LendingPool/LendingPool.sol
@@ -110,8 +110,10 @@ contract LendingPool is ILendingPool, LendingPoolStorage, Ownable,
ReentrancyGuard

    function claimProtocolFees() external {
        ReserveLibrary.updateReserveState(reserve, rateData);
+        _ensureLiquidity(reserve.pendingProtocolFeeAmount);
        (, uint256 claimedAmount) = LendingPoolFeeLibrary.claimProtocolFees(reserve,
feeCollector);
        ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, 0, claimedAmount);
+        _rebalanceLiquidity();
    }
    /**

    * @notice Collects a fee from the user
```



[L-02] `DebtToken._update()` emits duplicate `Transfer` event

Description

`DebtToken._update()` manually emits a `Transfer` event after calling `super._update()`, which already emits the same event in OpenZeppelin's `ERC20._update()`:

```
function _update(address from, address to, uint256 amount) internal virtual override {
    if (from != address(0) && to != address(0)) {
        revert TransfersNotAllowed();
    }
    super._update(from, to, amount); // emits Transfer(from, to, amount)
    emit Transfer(from, to, amount); // duplicate
}
```

Every `DebtToken` mint and burn emits `Transfer(from, to, amount)` twice. Any off-chain system relying on these events to track balances or transfers may compute incorrect values.

Remove the manual `emit Transfer(from, to, amount)` on the last line.

[L-03] `rescueToken` does not protect vault share or reserve asset

Description

`LendingPool.rescueToken()` only prevents rescuing `reserveRTokenAddress` (the `RToken`):

```
function rescueToken(address tokenAddress, address recipient, uint256 amount) external
onlyOwner {
    require(tokenAddress != reserve.reserveRTokenAddress, "Cannot rescue RToken");
    IERC20(tokenAddress).safeTransfer(recipient, amount);
}
```

Two critical assets are unprotected:

1. **Vault shares (scrVUSD):** Vault shares are held by the `LendingPool` contract. Rescuing vault shares transfers them out without updating `vaultRewards.totalShares`, permanently desynchronizing vault accounting. All subsequent vault operations (`_harvestYield`, `withdrawFromVault`, `ensureLiquidity`, `withdrawAll`) would revert or behave incorrectly.
2. **Reserve asset (crvUSD):** When `_harvestYield()` redeems yield shares, `crvUSD` is sent to the `LendingPool` and tracked as `vaultRewards.unclaimedRewards`. The owner can sweep this via `rescueToken`. The `unclaimedRewards` counter is not decremented, so `claimCollectorRewards()` would attempt to transfer `crvUSD` that no longer exists.

Consider adding the vault address and reserve asset to the rescue exclusion list.



[L-04] Mismatch between NatSpec and implementation for `liquidateBorrower` pause state

Description

The NatSpec for `StabilityPool.liquidateBorrower` states that the function should only be callable when the contract is not paused. However, the implementation lacks the `whenNotPaused` modifier. While allowing trusted managers to liquidate during a pause is often desirable for protocol solvency, the implementation contradicts the documented specification.

Consider updating the documentation to reflect that managers can liquidate even when the protocol is paused, OR add the `whenNotPaused` modifier if the restriction was intended.

[L-05] Missing event emission on setter function

Description

Several critical setter functions in the protocol do not emit events when state variables are updated. This hinders off-chain monitoring and indexing of protocol configuration changes.

Affected Functions:

- LiquidationSwap: `setLiquidityPool`, `setStabilityPool`, `setCrvusdToUSDOracle`, `setEnableSlippageProtection`.
- ERC20AssetAdapter: `setPriceOracle`.
- ERC721AssetAdapter: `setPriceOracle`.

Consider defining and emitting events for all state-changing parameter updates to ensure transparency and trackability.

[L-06] `claimProtocolFees` lacks access control

Description

The `LendingPool.claimProtocolFees` function has no access control modifier. Anyone can call this function at any time, forcing protocol fee collection. The `claimedAmount` is subtracted from `totalLiquidity` via `updateInterestRatesAndLiquidity`. An attacker can force-claim protocol fees at strategically bad moments, reducing `totalLiquidity` and spiking the utilization rate. While the fees are legitimately owed to the protocol, the timing of collection should be controlled to avoid liquidity shocks. Consider adding the `onlyFeeCollector` modifier ensuring the fee extraction happens at a predictable moment to avoid liquidity shocks.